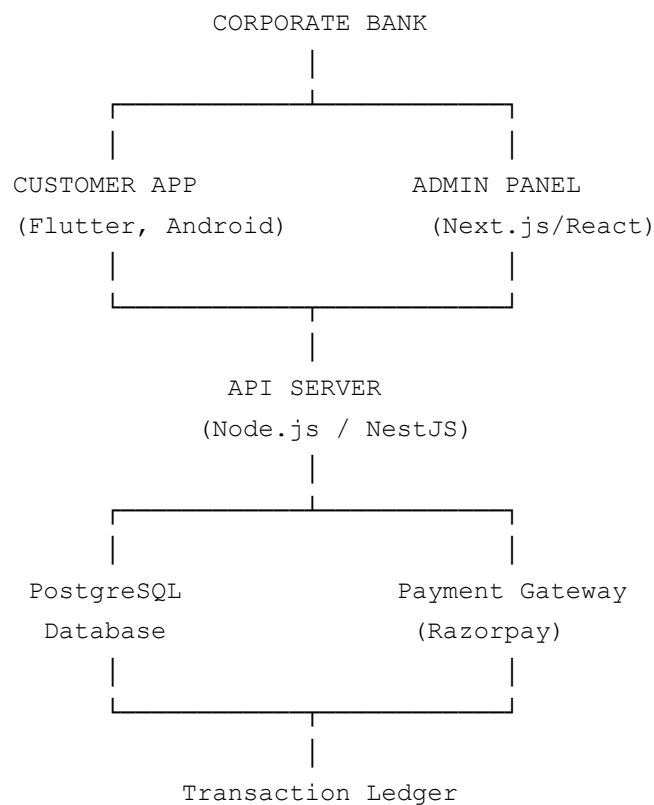


Digital Pigmee — Full Build Plan

Corporate Bank Digital Pigmy (Daily Micro-Savings) App

1. Architecture



Tech Stack

Layer	Choice
Customer App	Flutter (Android first)
Admin Panel	Next.js + React
Backend	Node.js + NestJS (REST)
Database	PostgreSQL
Auth	Mobile OTP + JWT + role-based access

2. Database Schema

villages

id, name, code, created_at

customers

id, village_id (FK), name, mobile, address, photo_url, kyc_status, created_at

nominees

id, customer_id (FK), name, relation, mobile, address

customer_documents

id, customer_id (FK), doc_type, file_url, verified_status, uploaded_at

customer_bank_details

id, customer_id (FK), account_number, ifsc, account_holder_name

pigmy_accounts

id, customer_id (FK), daily_amount, current_balance, total_deposited, status

transactions

id, pigmy_account_id (FK), amount, gateway_txn_id, status (pending/success/fail

ledger_entries

id, pigmy_account_id (FK), transaction_id (FK), type (credit/debit),
amount, previous_balance, new_balance, created_at

admins

id, name, email, password_hash, role, assigned_villages

audit_logs

id, actor_id, actor_type, action, entity, entity_id, before, after, created_at

Golden rule: `current_balance` is never directly writable. Every balance change happens through a `ledger_entries` row created by the ledger service. No manual overwrite, ever — this is what gives you the audit trail.

3. Backend (NestJS) — Modules

Module	Responsibility
auth	OTP send/verify, JWT issue/refresh, admin login, role guards
villages	CRUD, village-scoped data filtering
customers	Registration, profile, KYC, village assignment
pigmy	Pigmy account creation, balance queries
ledger	Core engine — deposit(), computeBalance(), all balance mutations
payments	Razorpay order creation, webhook handler, verification
reports	Date-wise / village-wise collection reports
audit	Write-only log of every sensitive action

Core transaction flow:

```

Customer taps Pay Now
  → API creates payment order (Razorpay)
  → Customer completes payment
  → Razorpay webhook hits /payments/webhook
  → Server verifies signature
  → transactions row created (status: success)
  → ledger service creates ledger_entries row
  → pigmy_accounts.current_balance recomputed/updated
  → receipt generated
  → admin dashboard reflects new numbers (next query, or push via websocket late

```

4. Customer App Screens

1. Splash Screen
2. Onboarding
3. Mobile Number Entry
4. OTP Verification
5. Registration (name, address, mobile)
6. Village Selection/Assignment

7. Customer Profile / KYC (photo, nominee, bank/payment details, documents)
8. Dashboard (balance card, today's pigmy, Pay Now)
9. Pigmy Account details
10. Payment Processing (gateway handoff)
11. Payment Success
12. Digital Receipt (downloadable/shareable)
13. Transaction History
14. Notifications
15. Help/Support
16. Logout

Dashboard layout reference:

Hello, Rahul 🙌 Village A

PIGMY BALANCE
₹ 12,500
Total Deposited
₹ 12,500

Today's Pigmy: ₹100

[PAY NOW]

Recent Transactions: ₹100 Successful ×3

5. Admin Panel Pages

1. Login
2. Dashboard (today's collection, total customers, success/pending/failed counts)
3. Today's Collection (dedicated drill-down view)
4. Village Management (list → drill into each village)
5. Customer Management (search, filter by village)
6. Customer 360° view (profile + pigmy account + full history + KYC + nominee + bank details in one screen)

7. Pigmy Accounts overview
8. Payment Transactions (all, filterable by status/date/village)
9. Pending Payments
10. Date-wise Reports
11. Village-wise Reports
12. Collection Analytics
13. Notifications (system/broadcast notifications to customers)
14. Audit Logs (read-only)
15. Admin Settings / role management

Village isolation: every admin query filters by `village_id` unless the admin role has multi-village access.

6. Security Requirements

- Balances are **derived, never directly editable** — enforced at the service layer, not just UI
 - Payment verification happens **server-side only**; client “success” callbacks are never trusted on their own
 - Webhook signature verification on every Razorpay callback
 - Idempotency keys on payment processing (prevent double-crediting on webhook retries)
 - Rate limiting on OTP requests
 - Role-based access control on all admin routes
 - Every balance-affecting action writes an `audit_logs` row
-

7. Build Order

1. Project structure (repos/monorepo, CI skeleton)
2. Database schema + migrations

3. Auth (OTP, JWT, admin login, role guards)
 4. Village + Admin base system
 5. Customer registration + profile system
 6. Pigmy account + Ledger engine (build and unit-test this in isolation before wiring UI to it)
 7. Payment gateway integration (sandbox first)
 8. Transaction ledger wired end-to-end
 9. Receipts + notifications
 10. Reports (date-wise, village-wise)
 11. Admin panel UI
 12. Customer app UI
 13. Security pass + audit log completeness check
 14. Testing (unit + end-to-end payment flow)
 15. Deployment
-

8. Deployment Targets (fast path)

- API: Railway or Render
 - Admin Panel: Vercel
 - Database: managed Postgres (Railway/Render/Supabase)
 - Customer App: APK build for direct install/testing (Play Store submission separately)
-

9. Design Direction

Customer App: premium fintech feel — clean dashboard, large balance card, prominent Pay Now button, modern transaction timeline, digital receipts, Hindi/English support architecture.

Admin Panel: professional corporate dashboard — village-wise cards, collection charts, searchable/filterable customer tables, date-range reports, clear payment status indicators.

